LABORATORIO 1 Y EXTENSIÓN RETO 1

Facturación con días hábiles

Erian Badilla Fallas · FWD Costa Rica
Programación Front End con IA Aplicada · 19 de octubre de 2026

Problema y propósito

Una empresa necesita conocer la fecha de vencimiento de sus facturas y detectar cuáles agotaron su plazo. Sumar días naturales puede producir errores si el periodo incluye fines de semana o feriados. FacturaLab CR consulta JSON Server y los feriados de Nager.Date, calcula el vencimiento y actualiza los registros pendientes mediante n8n.

La solución utiliza JavaScript puro, consola y un workflow, sin interfaz web. Los datos de facturas son ficticios y los feriados incluidos proceden de respuestas reales de la API. La extensión agrega dos políticas comerciales de vencimiento mediante herencia y polimorfismo, conservando el cálculo del laboratorio base.

Reglas comunes

El plazo contractual debe ser un entero entre uno y sesenta días hábiles. El conteo comienza después del día de emisión y excluye sábados, domingos y feriados nacionales de tipo Public reportados para Costa Rica. Se consultan el año de emisión y el siguiente, para permitir vencimientos en enero.

Una factura permanece pendiente durante su último día de plazo y pasa a vencida solo en una fecha posterior. Pagadas, canceladas y ya vencidas no se modifican. Se validan ID, cliente, monto positivo, país, estado y fechas reales; una emisión futura se rechaza. Los IDs repetidos dentro del lote se bloquean y los valores ya coincidentes no generan otro PATCH.

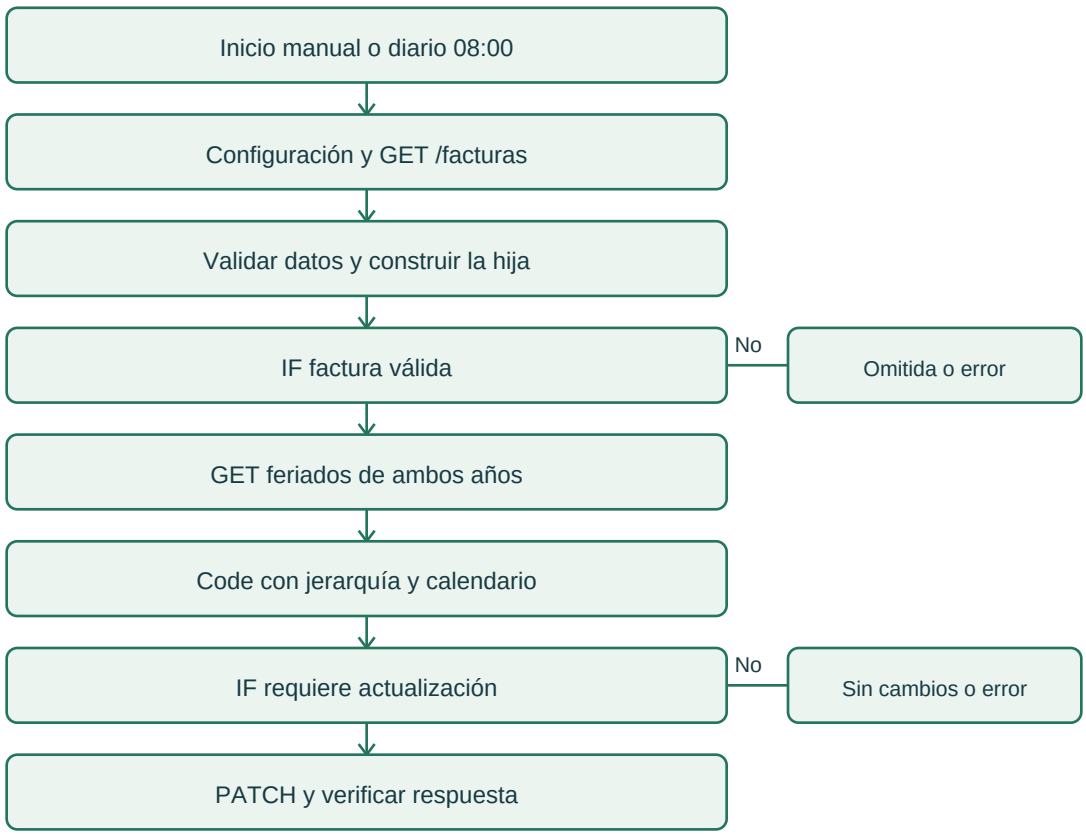
Alcance de la gracia comercial

Una factura estándar conserva exclusivamente el plazo contractual. Una factura con gracia agrega de uno a cinco días hábiles pactados al vencimiento contractual; por ello, el plazo efectivo puede llegar a sesenta y cinco días. La clasificación del atraso utiliza ese vencimiento efectivo.

Estas políticas pertenecen al ejercicio y no constituyen una interpretación de plazos legales de Costa Rica. No se calculan intereses, no se consulta Hacienda y no se emiten comprobantes electrónicos. Los montos de ejemplo se expresan en colones.

Recorrido y decisiones

El trigger manual sirve para demostrar la solución y el Schedule permite ejecutarla diariamente a las 08:00 de Costa Rica. El archivo se entrega inactivo; las consultas no necesitan API keys.



Datos relacionados con cada factura

El primer IF separa entradas inválidas y estados excluidos. Cada respuesta HTTP queda enlazada a su factura. Los dos calendarios se validan antes del cálculo y el segundo IF permite el PATCH únicamente si existen cambios. La nueva jerarquía vive en el mismo Code que procesa los registros estándar y con gracia.

Resultado y manejo de errores

Una fecha inválida, una gracia incompleta o un tipo desconocido producen un error visible sin escribir para ese registro. Si JSON Server no devuelve una colección o la API de feriados falla, no se inventan datos ni se guarda un vencimiento provisional. El timeout HTTP es de diez segundos.

Después del PATCH se comprueban ID, estado y vencimiento. Una respuesta no confirmada produce ESCRITURA_NO_CONFIRMADA y requiere consultar el registro antes de repetir: un timeout podría suceder después de haber guardado el cambio.

EXTENSIÓN SELECCIONADA

Herencia y polimorfismo

Se implementa el Reto 1 del documento de extensión, que permite elegir uno o varios retos y recomienda comenzar con uno. Factura es la clase base; FacturaEstandar y FacturaConGracia heredan de ella y sobreescriben el mismo método calcularVencimiento(calendario).

Clase	Responsabilidad y método
Factura	Valida datos y conserva el estado privado. evaluar llama a this.calcularVencimiento y decide el estado.
FacturaEstandar	Sobreescribe calcularVencimiento: suma únicamente los días hábiles contractuales a la emisión.
FacturaConGracia	Sobreescribe calcularVencimiento: obtiene el contractual con super y añade días hábiles de gracia. Valida gracia entera de 1 a 5.
CalendarioLaboral	Encapsula feriados y años disponibles. esDiaHabil y sumarDiasHabiles resuelven el calendario compartido.

Una llamada común con respuestas distintas

crearFactura selecciona la hija al construir el objeto. Después, el consumidor utiliza evaluar o calcularVencimiento sin preguntar por el tipo. JavaScript elige la implementación del objeto: ese despacho constituye el polimorfismo. Los registros anteriores sin tipo se interpretan como estándar.

Ambas facturas del siguiente ejemplo se emiten el 15 de octubre de 2026 con un día hábil contractual. Se evalúan el 19 de octubre:

Tipo	Vencimiento efectivo	Estado
Estándar	16 de octubre de 2026	Vencida
Con dos días hábiles de gracia	20 de octubre de 2026	Pendiente

La gracia no consume el sábado ni el domingo. El comando npm run demo:polimorfismo muestra los dos objetos bajo el mismo consumidor. FAC-005 lleva esta variante a JSON Server y n8n. Los constructores hijos utilizan super(datos), mientras que los campos privados y getters protegen los datos internos.

La relación entre factura y calendario sigue siendo composición. La herencia se usa solo donde existe una relación coherente: las dos variantes son facturas.

Una fuente de negocio

src/models/dominio.js contiene las clases y la fábrica. main.js evalúa instancias sin depender de n8n y demo-polimorfismo.js compara las hijas. build-workflow.js incorpora exactamente la fuente del dominio en los nodos Code al generar el JSON; no se mantiene un segundo algoritmo de fechas.

Ejemplo completo al 19 de octubre de 2026

Factura	Política	Vencimiento	Resultado
FAC-001	Estándar · 3 hábiles	16 sep 2026	Vencida
FAC-002	Estándar · 1 hábil	19 oct 2026	Pendiente
FAC-003	Estándar · 15 hábiles	9 nov 2026	Pendiente
FAC-004	Pagada	Sin cambio	Omitida
FAC-005	1 hábil + 2 de gracia	20 oct 2026	Pendiente

Pruebas y demostración

La suite tiene veintidós pruebas sobre cálculo, errores, duplicados, repetición, herencia, polimorfismo y equivalencia del Code generado con las clases. El procedimiento aislado importa el JSON en n8n 2.23.4, ejecuta sobre JSON Server temporal y comprueba los registros resultantes. Las salidas realmente obtenidas se conservan en evidencias/VERIFICACION.md y runtime.json.

La exposición debe mostrar el estado inicial y final, ejecutar otra vez para comprobar que no hay escrituras innecesarias y provocar una fecha inválida o un error de gracia. El estudiante todavía debe configurar la URL en su equipo, subir el repositorio y defender la solución en vivo.

Decisiones técnicas y límites

Se usan fechas YYYY-MM-DD recorridas en UTC y la fecha actual de Costa Rica para evitar cambios de día por conversión horaria. JSON Server 0.17.4 devuelve la colección completa sin paginación; para volumen alto se necesitarían filtros y caché anual. Los PATCH son deterministas, pero no hay protección transaccional contra un pago simultáneo ni garantía de ejecución exactamente una vez.

Fuentes: consigna del Laboratorio 1 y Laboratorio1_Extension.docx; date.nager.at/api/v3/PublicHolidays/2026/CR y /2027/CR; docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/; github.com/typicode/json-server/tree/v0.17.4.